

CS336 作业 1 (基础篇): 构建 Transformer 语言模型

版本: 26.0.3 授课团队: CS336 Staff 学期: 2026年春季

1. 作业概述 (Assignment Overview)

在本作业中, 你将从零开始构建训练标准 Transformer 语言模型 (LM) 所需的所有组件, 并训练一些模型。

你将实现的组件

1. BPE 分词器 (Byte-Pair Encoding Tokenizer) (第 2 节)
2. Transformer 语言模型 (LM) (第 3 节)
3. 交叉熵损失函数 (Cross-Entropy Loss) 与 AdamW 优化器 (第 4 节)
4. 训练循环 (Training Loop), 支持序列化与加载模型及优化器状态 (第 5 节)

你将运行的实验

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器对数据集进行编码, 将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer LM 进行样本生成与困惑度 (Perplexity) 评估。
5. 在 OpenWebText 上进行模型训练, 并将你达到的困惑度提交至排行榜。

你可以使用的工具与库

我们期望你从零开始构建每个组件。特别地, 除了以下允许的内容外, 你**不得使用** `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义:

- `torch.nn.Parameter`
- `torch.nn` 中的容器类 (例如 `Module`, `ModuleList`, `Sequential` 等)
- `torch.optim.Optimizer` 基类

你可以使用 PyTorch 的任何其他定义。如果你想使用某个函数或类但不确定是否被允许, 请随时在 Slack 上询问。如有疑问, 请思考使用它是否会损害本作业“从零构建 (from-scratch)”的精神。

关于 AI 工具的声明

AI 可以完全自主地解决作业的大部分内容。这使得深入接触和学习课程材料变得更加困难。

允许使用 AI 工具来回答高层概念性问题, 或提供低阶编程文档 (如函数签名和库 API)。但是, **不允许**使用 AI 工具来实现任何作业的任何部分。这既包括代码 Agent (如 Cursor Agents, Codex, Claude Code), 也包括 AI 自动补全 (如 Cursor Tab, GitHub Copilot)。使用 AI Agent 时, 请确保其使用提供的 `AGENTS.md` 文件。使用聊天机器人时也应包含 Prompt。

我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动补全 (尽管非 AI 的自动补全, 如函数名自动补全完全没有问题)。往届学生强调, 禁用 AI 自动补全更容易深入理解材料。完整 AI 政策请参阅相关文档。

代码结构

作业代码及本说明文档可在 GitHub 上获取: `github.com/stanford-cs336/assignment1-basics`。请 `git clone` 该仓库。如果有任何更新, 我们会通知你以便 `git pull` 获取最新内容。

1. `cs336_basics/*`: 这是你编写代码的地方。注意这里没有任何现成代码——你可以从零开始自由发挥!

2. `adapters.py`：你的代码必须具备一组特定的功能。对于每项功能（例如缩放点积注意力），只需通过调用你的代码来填写其实现（例如 `run_scaled_dot_product_attention`）。注意：对 `adapters.py` 的修改不应包含任何实质逻辑，这只是胶水代码。
3. `test_*.py`：包含你必须通过的所有测试（例如 `test_scaled_dot_product_attention`），它们将调用 `adapters.py` 中定义的钩子。请勿编辑测试文件。

如何提交

提交时，请运行 `make_submission.sh` 构建提交的 zip 文件。如果你有不希望包含在提交包中的大型数据文件或检查点，请确保将其添加到脚本的排除列表中。你将向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请排版你的回答。
- `code.zip`：包含你编写的所有代码。

若要提交到排行榜，请向 github.com/stanford-cs336/assignment1-basics-leaderboard 提交 PR。

数据集获取

本作业将使用两个预处理数据集：TinyStories (R. Eldan et al., 2023) 和 OpenWebText (A. Gokaslan et al., 2019)。两个数据集均为单个大型纯文本文件。如果你与课程同步做作业，可在计算指南中找到下载说明；如果在家自学，可以使用 `README.md` 中的命令下载。

低资源提示：初始化 (Low-Resource Tip: Init)

在整个课程的作业讲义中，我们将针对使用较少或没有 GPU 资源的情况提供建议。例如，我们有时会建议缩小数据集或模型规模，或解释如何在 Mac 集成 GPU 或 CPU 上运行训练代码。你将在蓝色框中找到这些“低资源提示”。即使你是能够使用课程机器的斯坦福在读学生，这些提示也能帮助你更快迭代并节省时间，因此建议阅读！

低资源提示：在 Apple Silicon 或 CPU 上运行作业 1

使用助教团队的参考代码，我们可以在配备 36 GB 内存的 Apple M4 Max 芯片上，使用 Metal GPU (MPS) 在 5 分钟内训练一个能生成相当流畅文本的 LM，使用 CPU 则约需 30 分钟。如果这些术语对你很陌生，别担心！只需知道，如果你有一台较新的笔记本电脑且实现正确高效，你就能训练一个生成简单儿童故事且流利度不错的小型 LM。稍后我们将解释在 CPU 或 MPS 上需要做哪些调整。

2. 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级字节对编码 (BPE) 分词器 (R. Sennrich et al., 2016; C. Wang et al., 2019)。具体来说，我们将任意 (Unicode) 字符串表示为字节序列，并在此字节序列上训练 BPE 分词器。稍后我们将使用该分词器将文本（字符串）编码为 Token（整数序列）以进行语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，将字符映射到整数码位 (Code Points)。截至 Unicode 17.0 (2025年9月发布)，该标准在 172 种剧本/脚本中定义了 159,801 个字符。例如，字符 "s" 的码位为 115（通常表示为 U+0073，其中 U+ 是传统前缀，0073 是 16 进制的 115），字符 “牛” 的码位为 29275。在 Python 中，可以使用 `ord()` 函数将单个 Unicode 字符转换为其整数表示，`chr()` 函数则将整数码位转换为对应的字符字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题 (unicode1): 理解 Unicode (1 分)

(a) `chr(0)` 返回什么 Unicode 字符?

交付物: 一句话回答。

(b) 该字符的字符串表示 (`repr()`) 与其打印表示 (`print()`) 有何不同?

交付物: 一句话回答。

(c) 当该字符出现在文本中时会发生什么? 在 Python 解释器中尝试以下代码:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付物: 一句话回答。

2.2 Unicode 编码

虽然 Unicode 标准定义了从字符到码位 (整数) 的映射, 但直接在 Unicode 码位上训练分词器是不切实际的, 因为词表过大 (约 15 万项) 且稀疏。相反, 我们使用 Unicode 编码将 Unicode 字符转换为字节序列。Unicode 标准定义了三种编码: UTF-8、UTF-16 和 UTF-32, 其中 UTF-8 是互联网的主导编码 (占有网页的 98% 以上)。

在 Python 中, 可使用 `encode()` 将 Unicode 字符串编码为 UTF-8; 迭代 `bytes` 对象可获取底层字节值; 使用 `decode()` 可将 UTF-8 字节串解码回 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x83\xe3\x81\x99!'
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 129, 175]
```

通过将 Unicode 码位转换为字节序列, 我们把 21 位整数 (159,801 个有效值) 转换为了字节值序列 (0 到 255 的整数)。256 长度的字节词表极易处理。使用字节级分词时, 无需担心未登录词 (OOV), 因为任何输入文本都可以表示为 0 到 255 之间的整数序列。

问题 (unicode2): Unicode 编码 (3 分)

(a) 相比 UTF-16 或 UTF-32, 选择在 UTF-8 编码的字节上训练分词器有哪些原因?

交付物: 一至两句话回答。

(b) 考虑以下错误的 UTF-8 解码函数。为什么该函数是错误的? 请提供一个产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
```

交付物: 一个导致错误的示例输入字节串, 以及一句话原因解释。

(c) 请给出一个无法解码为任何 Unicode 字符的双字节序列。

交付物: 示例及一句话解释。

2.3 子词分词 (Subword Tokenization)

字节级分词虽然解决了词级分词器的 OOV 问题, 但会导致输入序列变得极长。这会拖慢模型训练。子词分词是词级与字节级之间的折中方案。子词分词器通过更大的词表换取对输入字节序列更好的压缩。例如, 若字节序列 `b'the'` 频繁出现, 将其合并为一个词表项就能将 3 个 Token 压缩为 1 个。

在 BPE 算法中, 我们迭代地将最频繁出现的字节对替换 (合并) 为一个新的未使用索引。在本作业中, 我们将实现字节级 BPE 分词器。

2.4 BPE 分词器训练

BPE 训练包括三个主要步骤：

1. **词表初始化：** 初始词表为所有 256 个可能的字节值（大小为 256）。
2. **预分词 (Pre-tokenization)：** 为了防止跨标点符号（如 dog! 和 dog.）产生不必要的合并，我们先对话料库进行预分词。大多数现代分词器（如 GPT-2、tiktoken）使用基于正则表达式的预分词器。
3. **计算 BPE 合并：** 迭代统计每对连续字节的频率，选择最高频的字节对（"A", "B"）合并为新 Token "AB"。出现频率相同时，按照字典序较大的优先原则打破平局（例如 $\max([("A", "B"), ("BA", "A")])$ 选择（"BA", "A"））。

特殊 Token： 诸如 `<|endoftext|>` 的文本用于标识文档边界，必须作为不可分割的整体加入词表。

问题 (train_bpe)：BPE 分词器训练 (15 分)

编写一个给定输入文本路径训练字节级 BPE 分词器的函数。函数签名需包含 `input_path`, `vocab_size`, `special_tokens`, 并返回 `vocab: dict[int, bytes]` 和 `merges: list[tuple[bytes, bytes]]`。

问题 (train_bpe_tinystories)：在 TinyStories 上训练 BPE (2 分)

- (a) 在 TinyStories 上训练最大词表为 10,000 的 BPE 分词器（包含 `<|endoftext|>`）。训练消耗了多少时间和内存？词表中最长的 Token 是什么？是否合理？
- (b) 分析代码性能，哪一部分耗时最长？

问题 (train_bpe_expts_owt)：在 OpenWebText 上训练 BPE (2 分)

- (a) 在 OpenWebText 上训练 32,000 词表的 BPE 分词器。最长 Token 是什么？
- (b) 比较 TinyStories 与 OpenWebText 训练出的分词器的异同。

2.6 编码与解码 (Encoding and Decoding)

编码： 1. 预分词；2. 按训练时产生的合并规则顺序依次应用合并。

解码： 将整数 ID 映射回字节序列并拼接，使用 UTF-8 解码。若遇到非法字节序列，使用官方替换字符 `U+FFFD`（即 `errors='replace'`）。

问题 (tokenizer)：实现 Tokenizer 类 (15 分)

实现包含 `__init__`, `from_files`, `encode`, `encode_iterable`, `decode` 方法的 `Tokenizer` 类。

问题 (tokenizer_experiments)：分词器实验 (4 分)

- (a) 采样 10 篇文档计算压缩比 (bytes/token)。
- (b) 用 TinyStories 分词器去分词 OpenWebText 样本会发生什么？
- (c) 估计吞吐量（字节/秒），分词 The Pile 数据集 (825GB) 需要多久？
- (d) 将数据集序列化为 `uint16` 的 NumPy 数组，为什么 `uint16` 是合适选择？

3. Transformer 语言模型架构

语言模型输入形状为 `(batch_size, sequence_length)` 的 Token ID 张量，输出形状为 `(batch_size, sequence_length, vocab_size)` 的下一个 Token 的归一化概率分布。

Inputs → Token Embedding → [Transformer Block × num_layers] → Norm → Linear (LM Head) → Softmax
→ Output Probabilities

图 1: Transformer 语言模型整体架构

Input → Norm → Causal Multi-Head Self-Attention w/ RoPE → (+) Add → Norm → Position-Wise Feed-Forward → (+) Add → Output

图 2: Pre-Norm Transformer 块结构

3.2 数学符号与内存布局

在 PyTorch 中采用行主序 (Row-major)。线性变换表示为 $y = xW^T$ 。

3.3 基本构建块

- **参数初始化：** 线性层权重采用截断正态分布 $\mathcal{N}(\mu=0, \sigma^2 = 2/(d_{in}+d_{out}))$ ；嵌入层采用 $\mathcal{N}(0, 1)$ 截断于 $[-3, 3]$ ；RMSNorm 初始化为全 1。使用 `torch.nn.init.trunc_normal_`。

问题 (linear)：实现 Linear 模块 (1 分)

继承 `torch.nn.Module`，实现不带偏置 (bias) 的线性变换层。

问题 (embedding)：实现 Embedding 模块 (1 分)

继承 `torch.nn.Module`，实现嵌入查找层。

3.4 Pre-Norm Transformer 块组件

3.4.1 RMSNorm (Root Mean Square Layer Normalization)

$$RMSNorm(a)_i = (a_i / RMS(a)) \cdot g_i, \text{ 其中 } RMS(a) = \sqrt{(1/d_{model}) \sum a_i^2 + \epsilon}$$

注：在计算平方和前须将输入转换为 `torch.float32`，计算完成后转回原数据类型。

问题 (rmsnorm)：实现 RMSNorm (1 分)

实现 RMSNorm 模块。

3.4.2 位置级前馈网络 (SwiGLU)

现代 LLM 普遍采用 SwiGLU 激活函数：

$$FFN(x) = SwiGLU(x, W_1, W_2, W_3) = W_2(SiLU(W_1x) \odot W_3x)$$

其中 $d_{ff} \approx (8/3)d_{model}$ (通常向上取整为 64 的倍数)。

问题 (positionwise_feedforward)：实现前馈网络 (2 分)

实现 SwiGLU 前馈网络。

3.4.3 旋转位置嵌入 (RoPE)

给查询向量 $q^{(i)}$ 应用旋转矩阵 R_i ，对每对维度旋转角度 $\theta_{i,k} = i / \Theta^{(2k-2)/d}$ 。

问题 (rope): 实现 RoPE (2 分)

实现 `RotaryPositionalEmbedding` 类。

3.4.4 缩放点积注意力

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

问题 (softmax): 实现 Softmax (1 分)

实现数值稳定的 Softmax (减去最大值)。

问题 (scaled_dot_product_attention): 实现缩放点积注意力 (5 分)

实现支持 Mask 的缩放点积注意力。

问题 (multihead_self_attention): 实现因果多头自注意力 (5 分)

实现包含因果 Mask 及 RoPE 位置编码的多头自注意力机制。

问题 (transformer_block): 实现 Transformer 块 (3 分)

组装 Pre-Norm Transformer Block。

问题 (transformer_lm): 实现完整 Transformer LM (3 分)

组装完整的 Transformer 语言模型。

问题 (transformer_accounting): 模型资源核算 (5 分)

- (a) 计算 GPT-2 XL 配置下的参数量及加载模型所需的内存。
- (b) 列出前向传播所需的所有矩阵乘法及其总 FLOPs。
- (c) 哪部分消耗 FLOPs 最多?
- (d) 分析 GPT-2 Small/Medium/Large 下各组件 FLOPs 占比变化。
- (e) 将上下文长度增至 16,384 时，FLOPs 如何变化?

4. 训练 Transformer 语言模型

4.1 交叉熵损失与困惑度

问题 (cross_entropy): 实现交叉熵损失 (1 分)

实现数值稳定的交叉熵损失函数。

4.2 与 4.3 优化器 (SGD & AdamW)

问题 (learning_rate_tuning): 学习率调优 (1 分)

测试不同学习率在 SGD 下的收敛表现。

问题 (adamw): 实现 AdamW 优化器 (2 分)

实现带权重衰减的 AdamW 优化器。

问题 (adamw_accounting): AdamW 训练资源核算 (2 分)

(a) 分析 AdamW 运行时的峰值内存 (参数、激活值、梯度、优化器状态)。

(b) 计算 80GB 显存下 GPT-2 XL 支持的最大 Batch Size。

(c) 单步 AdamW 需要多少 FLOPs?

(d) 计算在单张 H100 GPU (50% MFU) 上训练 GPT-2 XL 所需的时间。

4.4 学习率调度与梯度裁剪

问题 (learning_rate_schedule): 实现带预热的余弦衰减调度 (1 分)

实现 Warmup + Cosine Annealing 调度函数。

问题 (gradient_clipping): 实现梯度裁剪 (1 分)

实现基于 L2 范数的梯度裁剪。

5. 训练循环 (Training Loop)

问题 (data_loading): 实现数据加载 (2 分)

实现从整数数组中采样 Batch 的数据加载器, 支持 `np.memmap` 大文件加载。

问题 (checkpointing): 实现检查点保存与加载 (1 分)

实现 `save_checkpoint` 与 `load_checkpoint`。

问题 (training_together): 整合训练脚本 (4 分)

编写包含完整训练、验证、日志记录与检查点保存的主训练脚本。

6. 文本生成 (Generating Text)

实现带温度采样 (Temperature Scaling) 和核采样 (Top-p / Nucleus Sampling) 的解码生成函数。

问题 (decoding): 实现文本解码生成 (3 分)

实现支持 Prompt、最大 Token 数、Temperature 和 Top-p 的解码函数。

7. 实验 (Experiments)

问题 (experiment_log): 实验日志记录 (3 分)

搭建实验追踪基础设施并记录训练曲线。

问题 (learning_rate): 调优学习率 (3 分)

在 TinyStories 上进行学习率 Scan，汇报验证集 Loss ≤ 1.45 的模型，并探讨“处于稳定边缘”的最佳学习率现象。

问题 (batch_size_experiment): Batch Size 变体实验 (1 分)

探索不同 Batch Size 对训练收敛及效率的影响。